

Project Proposal: X-Ray Motion Artifact Simulator

Abdelrahman Sayed, Ahmed Raafat, Salah Mohamed,
Anas Mohamed, Ahmed Adil
Team 6

1 Context

In the field of medical imaging technology, patient motion during X-ray acquisition remains a significant challenge. [1] Motion artifacts degrade image quality, obscure vital anatomical details, and can lead to misdiagnoses or the need for repeated exposures, which increases patient radiation dose. [1] Currently, there is a need for accessible, software-based educational tools that allow for the safe, interactive exploration of these imaging dynamics without the need for physical X-ray machinery. [1]

2 Purpose

The primary purpose of this project is to develop a Python-based simulator that models the physical and mathematical processes of X-ray acquisition. This tool is designed to demonstrate how patient motion translates into image artifacts and to provide a platform for evaluating various noise models and digital restoration techniques.

3 Requirements

- **Phantom Generation:** Ability to generate a 3D numerical human phantom.
- **Configuration Interface:** A graphical user interface (GUI) allowing configuration of target areas (Chest, Head), viewing directions (AP/PA/Lateral), motion parameters, exposure settings, noise models, and mitigation techniques.
- **Physics Engine:** Accurate modeling of X-ray projection using the Beer-Lambert law and temporal integration to simulate motion blur.
- **Noise Modeling:** Implementation of quantum (Poisson), electronic (Gaussian), and combined noise profiles.
- **Post-Processing:** Inclusion of image restoration algorithms, specifically Unsharp Masking and Richardson-Lucy (RL) Deconvolution.
- **Quantitative Metrics:** Real-time calculation and display of Signal-to-Noise Ratio (SNR) and Peak Signal-to-Noise Ratio (PSNR).

4 Implementation

4.1 Mathematics

The simulation is driven by several core mathematical models:

1. X-Ray Projection (Beer-Lambert Law) The transmission of X-rays through the body is calculated using the Beer-Lambert law. [2] The detected intensity $I(u, v)$ at pixel (u, v) is:

$$I(u, v) = I_0 \cdot \exp \left(- \int \mu(x, y, z) dl \right)$$

[2] In the discrete voxel space of the simulator, this is approximated as:

$$I_{discrete}(u, v) = \exp \left(- \sum_i \mu_i(u, v) \cdot \Delta l \right)$$

[2] (Note: I_0 is normalized to 1.0, μ is the linear attenuation coefficient at 70 keV, and Δl represents the voxel size of 0.30 cm).

2. Motion Artifacts (Temporal Integration) Motion is integrated over the exposure window T to create the final blurred signal:

$$P_{final}(u, v) = \frac{1}{T} \int_0^T P(u, v; t) dt \approx \frac{1}{N} \sum_{i=0}^{N-1} P(u, v; t_i)$$

[3] Displacement models include linear uniform drift ($d(t) = v \cdot t$) and sinusoidal periodic movements like breathing ($d(t) = A \cdot \sin(2\pi ft)$).

3. Detector Noise Models

- **Poisson (Quantum Noise):** $k \sim \text{Poisson}(N_0 \cdot I)$ [4]
- **Gaussian (Electronic Noise):** $\hat{I} = I + \mathcal{N}(0, \sigma^2)$ where $\sigma = \frac{1}{\sqrt{N_0}}$ [4]
- **Combined:** $\hat{I} = \frac{k+r}{N_0}$, incorporating baseline read-out noise where $r \sim \mathcal{N}(0, \sigma_{read}^2)$ and $\sigma_{read} = 0.01 \cdot \sqrt{N_0}$. [4]

4. Artifact Mitigation Techniques

- **Unsharp Masking:** $I_{mitigated} = I + 0.65 \cdot (I - (I * G_\sigma))$ [5]
- **Richardson-Lucy Deconvolution:** $u^{(k+1)} = u^{(k)} \cdot \left(\tilde{h} * \frac{d}{h * u^{(k)}} \right)$ [6]

5. Evaluation Metrics

- **SNR:** $\text{SNR (dB)} = 20 \cdot \log_{10} \left(\frac{\text{RMS}_{signal}}{\text{RMS}_{noise}} \right)$ [7]
- **PSNR:** $\text{PSNR (dB)} = 10 \cdot \log_{10} \left(\frac{MAX_I^2}{\text{MSE}} \right)$ [8]

4.2 Flow

The software architecture follows a linear pipeline replicating clinical acquisition:

1. **Phantom Generation:** Initializes the 3D mathematical model of the patient’s anatomy using a point cloud to provide an accurate representation of the anatomical structure of the human body.
2. **User Configuration:** Registers all user inputs from the GUI regarding geometry, motion, and noise.
3. **Simulation Loop:** Discretizes the exposure time into multiple sub-intervals. For each interval, the software calculates displacement, shifts the 3D volume, projects the 2D image via the Beer-Lambert law, and averages all frames to produce the motion-blurred image.
4. **Noise Addition:** Applies the configured Poisson and/or Gaussian noise models based on simulated photon flux.
5. **Mitigation:** Executes the selected filtering or deconvolution algorithms on the degraded image.
6. **Metrics & Display:** Computes SNR and PSNR, updating the interface to show a side-by-side comparison of the ideal static image, the corrupted image, and the final mitigated image.

5 Expected Results

Upon completion, the simulator will successfully generate three distinct visual outputs for any given test case: a perfect “ground truth” static projection, a realistic image degraded by the precise mathematical combination of user-defined motion and quantum/electronic noise, and a restored image post-mitigation. The software will output reliable quantitative metrics (SNR and PSNR) that allow users to objectively evaluate the efficacy of Unsharp Masking versus Richardson-Lucy Deconvolution under varying clinical parameters.

References

- [1] Articl.net, Artifacts in Radiography, 2024, <https://articl.net/diagnostic-radiology/artifacts-in-radiography>
- [2] arXiv, arXiv:1811.04102v1 [physics.ins-det], 2018, <https://arxiv.org/pdf/1811.04102>
- [3] SciSpace, Synthetic Shutter Speed Imaging, 2006, <https://scispace.com/pdf/synthetic-shutter-speed-imaging.pdf>
- [4] MDPI, Poisson–Gaussian Noise Analysis and Estimation for Low-Dose X-ray Images in the NSCT Domain, 2018, <https://www.mdpi.com/1424-8220/18/4/1019>
- [5] Wikipedia, Unsharp masking, 2024, https://en.wikipedia.org/wiki/Unsharp_masking
- [6] Wikipedia, Richardson–Lucy deconvolution, 2024, https://en.wikipedia.org/wiki/Richardson%E2%80%93Lucy_deconvolution
- [7] Wikipedia, Signal-to-noise ratio, 2024, https://en.wikipedia.org/wiki/Signal-to-noise_ratio
- [8] MathWorks, psnr - Peak signal-to-noise ratio (PSNR), 2024, <https://www.mathworks.com/help/images/ref/psnr.html>